Part 1 — Programming + Data Structures

Stage 1: Absolute Basics → Core Programming

How to use this

For every problem:

- First solve it **yourself**.
- Don't immediately search for the solution.
- If possible, solve it in **C** → **Java** → **Python**.
- When a problem says "without library function", actually avoid it.
- For later problems, reuse your own functions instead of rewriting code.
- Keep track of concepts you had to look up.

LEVEL 0 — Getting Comfortable

A. Input, Output & Variables

Q1 — Hello, User

Read a name and print:

Hello, <name>!

Do it in C, Java and Python.

Q2 — Basic Calculator

Read two integers and print:

- Sum
- Difference
- Product
- Integer quotient
- Remainder

Example:

Input:
17 5

Output:

Sum = 22
Difference = 12
Product = 85
Quotient = 3
Remainder = 2

Explore: integer vs floating-point division in all three languages.

Q3 — Temperature Converter

Read Celsius and convert it to Fahrenheit.

Then extend it:

- Celsius → Fahrenheit
- Fahrenheit → Celsius
- Celsius → Kelvin
- Kelvin → Celsius

Let the user choose the conversion.

Q4 — Swap Two Values

Swap two integers.

Then solve it in three ways:

1. Using a temporary variable
2. Without a temporary variable
3. Using a function

C: Pay attention to passing values to functions.

Java: Investigate why ordinary primitive arguments behave differently from what you might expect.

Python: Investigate tuple unpacking.

LEVEL 1 — Conditions

B. Basic Decision Making

Q5 — Even or Odd

Read an integer and determine whether it is even or odd.

Then modify it to:

- Positive/negative
- Even/odd
- Zero

Q6 — Largest of Two

Read two numbers and print the larger one.

Then:

> What should happen if both numbers are equal?

Handle that case explicitly.

Q7 — Largest of Three

Read three numbers and find the largest.

Solve using:

1. Nested `if`
2. Logical operators
3. A function

Q8 — Leap Year

Determine whether a year is a leap year.

Test:

1900
2000
2024
2025

Don't simply memorize the rule. Determine why `1900` and `2000` behave differently.

Q9 — Grade Calculator

Input marks for five subjects.

Calculate:

- Total

- Average
- Percentage
- Grade

For example:

90–100 → A
80–89 → B
70–79 → C
60–69 → D
<60 → F

Then add validation:

> What should happen if a mark is `-5` or `105`?

LEVEL 2 — Loops

C. Counting

Q10 — Print 1 to N

Read `N` and print:

1 2 3 4 ... N

Then print:

N N-1 N-2 ... 1

Q11 — Sum of N Numbers

Input:

N = 10

Output:

55

Then create:

sum(n)

and call it from your main program.

Q12 — Sum of Even Numbers

Given `N`, calculate:

2 + 4 + 6 + ... + N

Then create variations:

1. Sum of odd numbers
2. Sum of multiples of 3
3. Sum of numbers divisible by both 3 and 5

Q13 — Multiplication Table

Given `N`, print:

N × 1 = ...
N × 2 = ...
...
N × 10 = ...

Then allow the user to specify the ending multiplier.

LEVEL 3 — Number Problems

Now we begin the classic programming problems.

D. Digits

Q14 — Count Digits

Input:

123456

Output:

6

Handle:

0
5
-12345

Q15 — Sum of Digits

Input: 12345
Output: 15

Then:

Find the digital root.

Example:

9875
→ 9+8+7+5
→ 29
→ 2+9
→ 11
→ 1+1
→ 2

Q16 — Reverse a Number

Input: 12345
Output: 54321

Investigate what happens with:

1200
-123
0
100000

Q17 — Number Palindrome

Determine whether:

121 → palindrome
123 → not palindrome

Do it **without converting the number to a string**.

Then later we'll compare it with the string-based approach.

Q18 — Armstrong Number

Determine whether a number is an Armstrong number.

Start with the traditional 3-digit definition:

$153 = 1^3 + 5^3 + 3^3$

Then don't stop there.

Modify your program so that it works for **any number of digits**.

LEVEL 4 — Functions

Now the important change:

Stop writing everything inside `main`.

Q19 — Number Utility Functions

Create separate functions:

isEven()
isOdd()
isPrime()
isPalindrome()
reverseNumber()
sumOfDigits()
countDigits()

Create a program that uses them.

Q20 — Number Analyzer

Input:

123456

Produce:

Number: 123456
Digits: 6
Digit Sum: 21
Reversed: 654321
Even: Yes
Palindrome: No
Prime: No

The important part:

> Each property must be calculated by a separate function.

Q21 — Function Arguments

Create:

power(base, exponent)

Then investigate:

power(2, 3)
power(5, 0)
power(0, 5)
power(-2, 3)

Then modify it so that the exponent can be negative.

Q22 — Multiple Arguments

Create:

maximum(a, b, c)
minimum(a, b, c)
average(a, b, c)

Then create a generalized version capable of handling an arbitrary number of values where the language allows a natural mechanism for doing so.

This is where we'll deliberately explore:

- C's approach
- Java's approach
- Python's approach

LEVEL 5 — Classic Patterns

Now we introduce problems that you'll repeatedly encounter throughout programming.

E. Fibonacci

Q23 — Fibonacci Series

Print the first N Fibonacci numbers.

Input: 8

Output:
0 1 1 2 3 5 8 13

Q24 — Fibonacci nth Term

Find only:

F(0)

F(1)
F(2)
...
F(N)

Q25 — Fibonacci Using a Function

Create:

fibonacci(n)

Compare:

1. Iterative solution
2. Recursive solution

Don't worry about optimization yet.

Q26 — Fibonacci Properties

Given `N`, determine:

- Whether `N` is a Fibonacci number
- Its position if it is
- The first Fibonacci number greater than `N`

LEVEL 6 — Pattern Printing

F. Star Patterns

Q27

For `N = 5`:

```
*
**
***
****
*****
```

Q28

```
*****
****
***
```

```
**
*
```

Q29

```
   *
   **
  ***
 ****
*****
```

Q30

```
*
***
*****
*******
```

Q31

```
   *
   ***
  *****
 *******
*********
```

Q32 — Number Triangle

```
1
12
123
1234
12345
```

Q33 — Repeated Number Triangle

```
1
22
333
4444
55555
```

Q34 — Floyd's Triangle

```
1
2 3
4 5 6
7 8 9 10
```

Q35 — Pattern Challenge

Given `N`, generate:

1
121
12321
1234321
123454321

Then reverse the pattern.

LEVEL 7 — Strings Begin

Now we switch from numbers to **characters, strings and words**.

G. Basic Strings

Q36 — Read and Print a String

Read a complete sentence and print it.

Test:

Hello World
I am learning programming
C programming is interesting

Pay attention to how C, Java and Python read strings containing spaces.

Q37 — String Length

Find the length of a string.

First:

> Don't use the language's built-in length function.

Then use the built-in function.

Compare the approaches.

Q38 — Reverse a String

Input:
hello

Output:

olleh

Solve it:

1. Without library functions
2. Using a character array/list
3. Using the language's string facilities

Q39 — String Palindrome

madam → palindrome
hello → not palindrome

Test:

racecar
level
A
aa
ab

Then investigate:

> Should `Madam` be considered a palindrome?

Make the decision yourself and modify your program accordingly.

Q40 — Count Characters

Given:

Hello World

calculate:

Letters = ?
Digits = ?
Spaces = ?
Special characters = ?

Q41 — Character Frequency

Input:

programming

Output something like:

p → 1
r → 2

o → 1
g → 2
a → 1
m → 2
i → 1
n → 1

Then modify it to print only characters occurring more than once.

LEVEL 8 — Words

Q42 — Count Words

Input:

I love programming in C

Output:

5

Test:

Hello
Hello World
  Hello   World
Hello World

This will force you to think about **whitespace**.

Q43 — Reverse Words

Input:
I love programming

Output:
programming love I

Don't reverse the characters within each word.

Q44 — Reverse Every Word

Input:
I love programming

Output:
I evol gnimmargorp

Q45 — Find the Longest Word

Input:
I am learning programming

Output:
programming

Then handle ties.

Q46 — Word Frequency

Input:

this is a test this is another test

Produce the frequency of every word.

Now explore:

- Case sensitivity
- Punctuation
- Multiple spaces

LEVEL 10 — Arrays / Collections

Now we move into actual data-structure thinking.

Q50 — Array Basics

Given an array:

[10, 20, 30, 40, 50]

implement:

- Print all elements
- Sum
- Average
- Minimum
- Maximum
- Search
- Count occurrences

Q51 — Reverse an Array

Reverse:

[1,2,3,4,5]

without creating another array.

Q52 — Second Largest

Find the second-largest element.

Then consider:

[5, 5, 5]
[5, 5, 3]
[1]
[-1,-2,-3]

What should your program return?

Q53 — Remove Duplicates

Given:

[1,2,2,3,4,4,5]

produce:

[1,2,3,4,5]

First solve it without using a set.

Then solve it using the language's appropriate collection.

Q54 — Frequency Table

Given:

[1,2,2,3,3,3,4]

produce:

1 → 1
2 → 2
3 → 3
4 → 1

Solve it using the natural mechanism of:

- C
- Java
- Python

This will introduce the different approaches to **hashing/maps/dictionaries**.

Where we go next

After these foundations, the progression becomes:

Level 11 — Searching

→ Linear search
→ Binary search
→ variations and edge cases

Level 12 — Sorting

→ Bubble
→ Selection
→ Insertion
→ then compare their behavior

Level 13 — Recursion

→ factorial
→ Fibonacci
→ reverse
→ digit problems
→ arrays
→ strings

Level 14 — Linked Lists

→ creation
→ traversal
→ insertion
→ deletion
→ reversal
→ variations

Level 15 — Stack

Level 16 — Queue

Level 17 — Hashing

Level 18 — Trees/BST

Level 19 — Heaps

Level 20 — Basic Graphs

And **alongside these**, we'll progressively branch into:

> **C → pointers → dynamic memory → structures → function pointers**
>
> **Java → classes → inheritance → polymorphism → interfaces → abstraction → composition → OOD**
>
> **Python → classes → properties → inheritance → multiple inheritance → ABCs → dunder methods → operator overloading → MRO → composition → iterators/context managers/dataclasses**

The important rule will be: **we don't jump from "write Fibonacci" to "implement a graph algorithm." Every level should force you to discover one or two new concepts while reusing everything you've already learned.**

LEVEL 9 — Library Exploration

Here we change the philosophy.

Instead of me telling you:

> "Use function X."

I'll give you a problem where **you have to discover which library function is appropriate**.

Q47 — C String Investigation

Solve a series of small cases requiring you to investigate:

- `strlen`
- `strcpy`
- `strncpy`
- `strcmp`
- `strncmp`
- `strcat`
- `strncat`
- character functions such as `toupper`, `tolower`, `isdigit`, etc.

For every function, determine:

1. What arguments does it take?
2. What does it return?
3. Does it modify the original data?
4. What happens with an empty string?
5. What happens with insufficient buffer space?
6. What happens with `NULL` where applicable?

Q48 — Java String Investigation

Without being told which method to use, solve cases involving:

- length
- character access
- comparison
- substring extraction
- searching
- replacement
- splitting
- trimming/whitespace
- case conversion
- conversion between strings and other representations

For every method, record its:

method
arguments
return type
effect
edge cases

Q49 — Python String Investigation

Solve cases requiring you to discover appropriate methods for:

- searching
- replacing
- splitting
- joining
- stripping
- changing case
- testing character types
- formatting
- prefix/suffix testing

Then compare Python's approach with C and Java.

https://chatgpt.com/share/6ac68996-8c4c-83ec-a0de-4bc599fb75aa